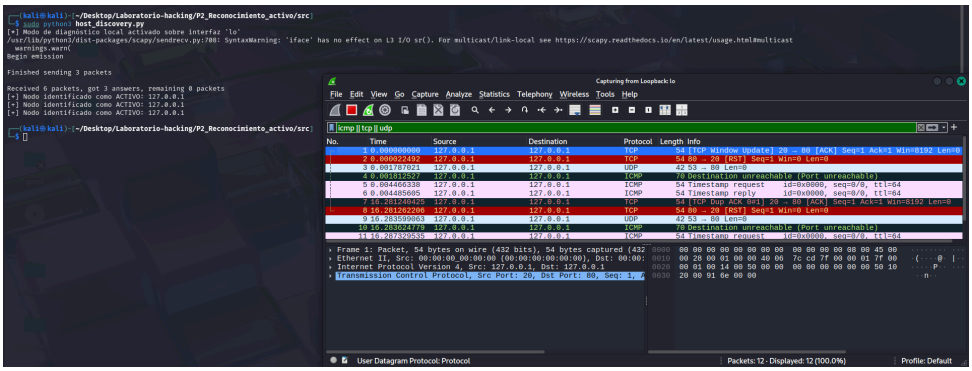
Práctica 2: Reconocimiento Activo

Técnicas de Hacking

Autor: Sergio Garcia Gallego-Casilda
Fecha: 21 de Abril de 2026
Profesor: Alfredo Robledano Abasolo
Asignatura: Técnicas de Hacking
Grado: Ingeniería de la Ciberseguridad
Universidad: Universidad Europea de Madrid



Captura Wireshark: tráfico ICMP, TCP y UDP durante el reconocimiento activo

Índice

1. Resumen ... 4
2. Introducción y Objetivos ... 5
3. Fundamentos Teóricos ... 6
   3.1. Reconocimiento Activo en Capas 3 y 4 ... 6
   3.2. Protocolo ICMP y Mensajes de Control ... 6
   3.3. Protocolo TCP y Flags de Control ... 6
   3.4. Protocolo UDP ... 6
4. Parte 1: Descubrimiento de Hosts con Scapy ... 7
   4.1. Diseño y Arquitectura de la Solución ... 7
      4.1.1. Parámetros de `craft_discovery_pkts` ... 7
      4.1.2. Vectores de Descubrimiento Implementados ... 7
   4.2. Código Fuente Completo ... 7
      4.2.1. Decisiones de Diseño Clave ... 8
   4.3. Evidencias de Ejecución ... 8
      4.3.1. Host Activo: `127.0.0.1` ... 8
      4.3.2. Host Inactivo: IP sin host asociado ... 9
5. Parte 2: Comportamiento por Defecto de Nmap ... 10
   5.1. Estado de Puerto: Definición y Estímulos ... 10
   5.2. Comportamiento por Defecto de Nmap ... 10
      5.2.1. Conteo de paquetes por puerto ... 10
   5.3. Entorno de Pruebas ... 10
   5.4. Evidencias: Puerto Abierto 8080/tcp (HTTP) ... 11
   5.5. Evidencias: Puerto Abierto 22/tcp (SSH) ... 12
   5.6. Evidencias: Puerto Cerrado 9999/tcp ... 12
   5.7. Comparativa de los Tres Estados ... 13
   5.8. Evidencias: Puerto Cerrado 80/tcp ... 13
   5.9. Evidencias: Puerto Cerrado 1234/tcp ... 14
   5.10. Evidencias: Puerto Filtrado 12345/tcp ... 15
   5.11. Resumen del Comportamiento por Defecto ... 16
6. Conclusiones ... 18
7. Referencias Bibliográficas ... 18

Índice de Figuras

Figura 1 Ejecución completa sobre host activo 127.0.0.1. Terminal izquierda: Scapy envía 3 paquetes y obtiene 3 respuestas, imprimiendo tres veces el nodo como ACTIVO. Wireshark derecha: RST (rojo, TCP ACK), ICMP Port Unreachable (celeste, UDP) e ICMP Timestamp Reply (rosa, ICMP_TS). 9

Figura 2 Ejecución contra IP inactiva: 0 respuestas obtenidas de 3 envíos. Los tres paquetes quedan en la lista `unanswered`, indicando ausencia de host o filtrado total del tráfico en ese segmento de red. 9

Figura 3 Resumen de los tres escaneos ejecutados en secuencia desde la terminal. Nmap 7.98 identifica `8080/tcp open http-proxy`, `22/tcp open ssh` y `9999/tcp closed abyss`, demostrando los tres estados posibles de puerto en un único entorno de pruebas controlado. 11

Figura 4 Evidencia Nmap — puerto **8080/tcp abierto**: secuencia `SYN → SYN/ACK → RST`. Terminal: `sudo nmap -sS -p 8080 127.0.0.1` ⟶ estado `open http-proxy`. Wireshark (filtro `tcp.port == 8080`): 3 paquetes. El SYN/ACK generado por el servidor HTTP Python en el paquete 2 es el indicador definitivo del estado **open**. Nmap cierra el intercambio con RST en el paquete 3, sin completar el handshake. 12

Figura 5 Evidencia Nmap — puerto **22/tcp abierto**: secuencia `SYN → SYN/ACK → RST`. Terminal: `sudo nmap -sS -p 22 127.0.0.1` ⟶ estado `open ssh`. Wireshark (filtro `tcp.port == 22`): 3 paquetes. El comportamiento es idéntico al del puerto 8080, evidenciando que el patrón SYN/ACK es universal para cualquier servicio activo, independientemente del protocolo de aplicación. 12

Figura 6 Evidencia Nmap — puerto **9999/tcp cerrado**: secuencia `SYN → RST/ACK`. Terminal: `sudo nmap -sS -p 9999 127.0.0.1` ⟶ estado `closed abyss`. Wireshark (filtro `tcp.port == 9999`): solo 2 paquetes. La ausencia de SYN/ACK es el indicador definitivo de que ningún proceso escucha en ese puerto. El RST/ACK es generado directamente por el kernel, no por ninguna aplicación. . . 13

Figura 7 Evidencia Nmap — puerto **80/tcp cerrado**: secuencia `SYN → RST/ACK`. Terminal: `sudo nmap -sS -p 80 127.0.0.1` ⟶ estado `closed`. Wireshark: 2 paquetes. El RST/ACK es generado directamente por el kernel al no existir servicio en el puerto 80, con latencia de apenas 60 µs. . 14

Figura 8 Evidencia Nmap — puerto **1234/tcp cerrado**: secuencia `SYN → RST/ACK`. Puerto no estándar que confirma la generalidad del comportamiento. Nmap etiqueta el servicio como `hotline` según su base de datos `nmap-services`. La latencia de respuesta RST es igualmente sub-milisegundo, gestionada íntegramente por el kernel sin intervención de ninguna aplicación en espacio de usuario. 14

Figura 9 Evidencia Nmap — puerto **12345/tcp filtrado**: secuencia `SYN → (timeout)`. Terminal: `sudo nmap -sS -p 12345 --max-retries 0 127.0.0.1` ⟶ estado `filtered netbus`. La duración de 1.06 s frente a los 0.08 s de un puerto cerrado evidencia el coste temporal del timeout. El aviso *«giving up on port because retransmission cap hit (0)»* confirma la ausencia total de respuesta. . 16

Figura 10 Wireshark — puerto **12345/tcp filtrado**: filtro `tcp.port == 12345`. Un único paquete SYN visible (paquete 1, sin respuesta), frente a los 3 paquetes del estado **open** y los 2 del estado **closed**. La ausencia de cualquier respuesta es la firma inequívoca del filtrado por firewall. 16

1. Resumen

Este trabajo documenta una práctica de seguridad informática en la que se han utilizado herramientas de análisis de red para descubrir dispositivos activos y estudiar el comportamiento de un escáner de puertos ampliamente usado en la industria.

En la primera parte se ha desarrollado un programa en Python capaz de enviar tres tipos distintos de mensajes de red a una dirección IP y determinar, en función de las respuestas recibidas, si hay un dispositivo activo en esa dirección. Se ha comprobado su funcionamiento tanto contra un equipo real como contra una dirección sin ningún dispositivo asociado.

En la segunda parte se ha analizado cómo funciona por defecto la herramienta Nmap, uno de los escáneres de red más utilizados en auditorías de seguridad. Se ha estudiado qué ocurre a nivel de tráfico de red cuando un puerto está abierto, cerrado o bloqueado por un cortafuegos, documentando cada caso con capturas reales de tráfico.

Todas las pruebas se han realizado en un entorno controlado sobre la propia máquina de trabajo, sin generar tráfico hacia sistemas externos, cumpliendo con los requisitos éticos y legales de la práctica.

2. Introducción y Objetivos

Este informe técnico documenta la implementación de herramientas personalizadas para el descubrimiento de activos en red y el análisis del comportamiento de la herramienta **Nmap**. El objetivo principal, según el enunciado de la práctica, es doble:

- **Parte 1:** Implementar estímulos adicionales a los vistos en clase para el descubrimiento de hosts, construyendo una función Python con Scapy que combine los protocolos **UDP**, **TCP (ACK)** e **ICMP (Timestamp)**.
- **Parte 2:** Definir y evidenciar el funcionamiento por defecto de Nmap en el descubrimiento de puertos y sus estados (abierto, cerrado, filtrado).

El trabajo refuerza competencias en el modelo OSI (capas 3 y 4), interpretación de flags TCP, manipulación de paquetes a bajo nivel y documentación técnica de auditorías de red.

3. Fundamentos Teóricos

3.1. Reconocimiento Activo en Capas 3 y 4

El reconocimiento activo opera principalmente en la **capa de red (L3)** y la **capa de transporte (L4)** del modelo OSI. En L3 se trabaja con el protocolo IP y los mensajes de control ICMP; en L4 se manipulan segmentos TCP y datagramas UDP. A diferencia del reconocimiento pasivo, el activo **genera tráfico hacia el objetivo**, lo que lo hace detectable pero también más preciso.

3.2. Protocolo ICMP y Mensajes de Control

ICMP (RFC 792) es el protocolo de mensajería de la capa de red. Los tipos relevantes para esta práctica son:

Tipo	Código	Descripción
8	0	Echo Request — ping estándar
0	0	Echo Reply — host activo
13	0	Timestamp Request — sincronización temporal (usado en esta práctica)
14	0	Timestamp Reply — respuesta al Timestamp Request
3	3	Port Unreachable — puerto UDP sin servicio (confirma host activo)
3	13	Administratively Prohibited — filtrado por firewall

3.3. Protocolo TCP y Flags de Control

TCP (RFC 793) es un protocolo orientado a conexión. Los flags más relevantes para el escaneo son:

Flag	Significado en contexto de escaneo
SYN	Solicitud de inicio de conexión
ACK	Acuse de recibo. En TCP ACK scan fuerza RST del host si está activo
RST	Reset: el host rechaza o cierra la conexión abruptamente
SYN/ACK	Puerto **abierto**: servicio a la escucha
RST/ACK	Puerto **cerrado**: host vivo, sin servicio en ese puerto

3.4. Protocolo UDP

UDP (RFC 768) no tiene conexión ni handshake. Si el puerto está **cerrado**, el host responde con ICMP tipo 3, código 3 (**Port Unreachable**). Si está **abierto**, normalmente no hay respuesta. Esto hace que su escaneo sea más lento y ambiguo que TCP.

4. Parte 1: Descubrimiento de Hosts con Scapy

4.1. Diseño y Arquitectura de la Solución

Se ha desarrollado una herramienta en Python utilizando el framework **Scapy**, que permite construir y manipular paquetes de red a nivel de librería con control total sobre cada campo de las cabeceras de protocolo. La solución se estructura en dos funciones:

- `craft_discovery_pkts`: generador de tráfico polimórfico que construye los paquetes de sondeo.
- `ejecutar_reconocimiento`: función principal que invoca la anterior, envía los paquetes con `sr()` y clasifica los resultados.

4.1.1. Parámetros de `craft_discovery_pkts`

Parámetro	Tipo	Descripción
`metodos`	Obligatorio	Lista o string con hasta 3 protocolos: `"TCP_ACK"`, `"UDP"`, `"ICMP_TS"`
`objetivos`	Obligatorio	IP en formato string o lista de IPs
`conteo`	Opcional	Dict `{protocolo: n_paquetes}`. Por defecto 1 paquete por protocolo
`puerto_base`	Opcional	Puerto destino para TCP y UDP. Por defecto 80

4.1.2. Vectores de Descubrimiento Implementados

- **TCP ACK Scanning:** Envía segmentos con flag `A` (ACK) activo. Los hosts activos responden con `RST` porque no existe sesión previa. Eficaz contra firewalls *stateless* que no filtran paquetes ACK.
- **UDP Probing:** Envía datagramas al puerto especificado. Si el puerto está cerrado, el host responde con ICMP Port Unreachable, confirmando su presencia en la red.
- **ICMP Timestamp (tipo 13):** Sonda de sincronización temporal. Más sigilosa que el Echo Request (tipo 8), ya que muchos firewalls bloquean el ping estándar pero no filtran los mensajes de timestamp.

El uso simultáneo de los tres protocolos aumenta la probabilidad de detección frente a políticas de filtrado heterogéneas: un firewall puede bloquear ICMP Echo pero permitir ACK o UDP. La combinación reduce los falsos negativos.

4.2. Código Fuente Completo

```python
from scapy.all import IP, TCP, UDP, ICMP, sr1, sr, srp, Ether, conf
import logging

logging.getLogger("scapy.runtime").setLevel(logging.ERROR)

def craft_discovery_pkts(metodos, objetivos, conteo=None, puerto_base=80):
    if isinstance(metodos, str): metodos = [metodos]
    lista_ips = [objetivos] if isinstance(objetivos, str) else objetivos
    conteo = conteo or {m: 1 for m in metodos}
    pool_paquetes = []

    for direccion_ip in lista_ips:
        encabezado_ip = IP(dst=direccion_ip)
        for modo in metodos:
            iteraciones = conteo.get(modo, 1)
            for _ in range(iteraciones):
                id_modo = modo.upper()
                if id_modo == "TCP_ACK":
```

```
                    pqt = encabezado_ip / TCP(dport=puerto_base, flags="A")
                elif id_modo == "UDP":
                    pqt = encabezado_ip / UDP(dport=puerto_base)
                elif id_modo == "ICMP_TS":
                    pqt = encabezado_ip / ICMP(type=13)
                else:
                    continue
                pool_paquetes.append(pqt)
    return pool_paquetes

def ejecutar_reconocimiento():
    nodos_objetivo = ["127.0.0.1"]
    tecnicas = ["TCP_ACK", "UDP", "ICMP_TS"]

    print(f"[*] Modo de diagnóstico local activado sobre interfaz 'lo'")
    trafico_generado = craft_discovery_pkts(tecnicas, nodos_objetivo)

    # sr() opera en capa 3 (IP), adecuado para la interfaz loopback
    respondidos, _ = sr(trafico_generado, timeout=2, verbose=1, iface="lo")

    if respondidos:
        for _, res in respondidos:
            print(f"[+] Nodo identificado como ACTIVO: {res.src}")
    else:
        print("[-] No se recibieron respuestas. Revisa Wireshark en interfaz 'lo'.")

if __name__ == "__main__":
    ejecutar_reconocimiento()
```

4.2.1. Decisiones de Diseño Clave

El uso de `sr()` frente a `srp()` se justifica porque operamos en **capa 3 (IP)**: `sr()` trabaja con raw IP packets sin necesidad de gestionar cabeceras Ethernet. `srp()` sería adecuado para capa 2 (p. ej. ARP). El parámetro `iface="lo"` es obligatorio para forzar el uso de la interfaz loopback y que las capturas de Wireshark sean coherentes con la ejecución.

La doble gestión `isinstance(metodos, str)` e `isinstance(objetivos, str)` permite invocar la función de forma flexible, tanto con un único string como con una lista, mejorando la ergonomía sin romper la compatibilidad con el enunciado.

4.3. Evidencias de Ejecución

4.3.1. Host Activo: `127.0.0.1`

El script identificó correctamente el host `127.0.0.1` como activo. Scapy reporta: *«Finished sending 3 packets / Received 6 packets, got 3 answers»*. El análisis en Wireshark confirma una respuesta para cada uno de los tres estímulos.

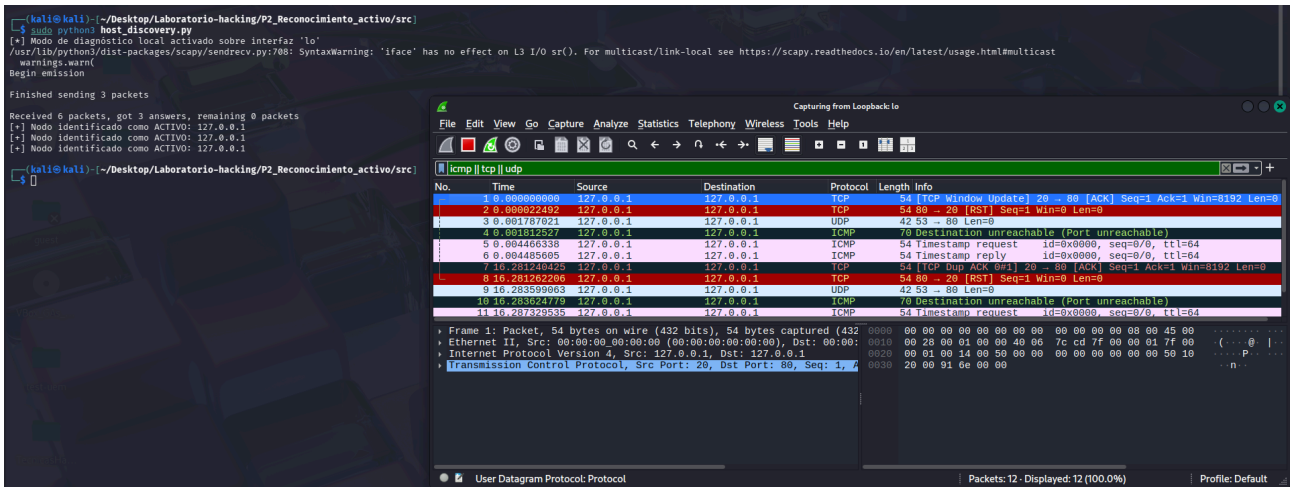


Figura 1: Ejecución completa sobre host activo 127.0.0.1. Terminal izquierda: Scapy envía 3 paquetes y obtiene 3 respuestas, imprimiendo tres veces el nodo como ACTIVO. Wireshark derecha: RST (rojo, TCP ACK), ICMP Port Unreachable (celeste, UDP) e ICMP Timestamp Reply (rosa, ICMP_TS).

Detalle de cada respuesta observada en Wireshark:

- **TCP RST (rojo, paquete 2):** Al recibir un ACK sin sesión previa, la pila TCP del kernel responde con RST. Confirma que el host está vivo aunque no haya servicio activo en el puerto destino.
- **ICMP Port Unreachable (celeste, paquete 4):** La sonda UDP llega al host, que no tiene servicio en 80/udp y genera ICMP tipo 3 código 3. El payload incluye los primeros 8 bytes del datagrama UDP original, permitiendo correlacionar respuesta con sonda.
- **ICMP Timestamp Reply (rosa, paquete 6):** El host responde con tipo 14. El campo *Originate Timestamp* del request es devuelto en la respuesta, confirmando la identidad del host.

4.3.2. Host Inactivo: IP sin host asociado

Se validó la función contra una IP sin host en el segmento local. El script gestionó el timeout correctamente: *«Received 0 packets, got 0 answers, remaining 3 packets»*.

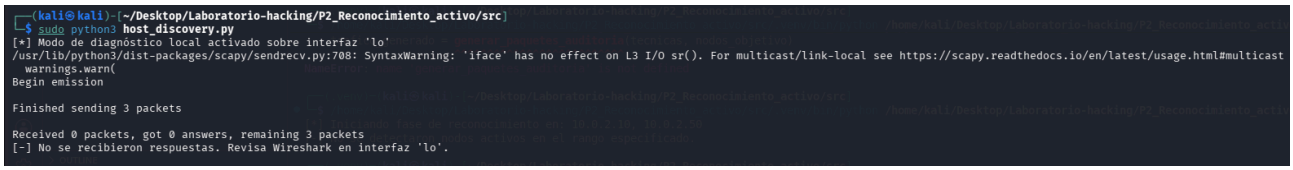


Figura 2: Ejecución contra IP inactiva: 0 respuestas obtenidas de 3 envíos. Los tres paquetes quedan en la lista `unanswered`, indicando ausencia de host o filtrado total del tráfico en ese segmento de red.

5. Parte 2: Comportamiento por Defecto de Nmap

5.1. Estado de Puerto: Definición y Estímulos

Un **estado de puerto** es la clasificación que asigna un escáner a un puerto TCP/UDP en función de la respuesta (o ausencia de ella) ante un estímulo concreto. Los tres estados principales son:

Estado	Estímulo	Respuesta recibida
open	SYN	SYN/ACK — servicio activamente a la escucha
closed	SYN	RST/ACK — host vivo, ningún servicio en ese puerto
filtered	SYN	Sin respuesta (timeout) o ICMP tipo 3 cód. 13 — firewall descarta el tráfico

5.2. Comportamiento por Defecto de Nmap

Al ejecutar `sudo nmap <target>` sin flags adicionales, Nmap:

1. **Selecciona los 1000 puertos más frecuentes** definidos en `nmap-services`, no los 1000 primeros numéricamente.
2. **Utiliza TCP SYN Stealth Scan** (`-sS`) si tiene privilegios de root. Envía un SYN y, tras recibir SYN/ACK, responde con RST sin completar el handshake. La conexión nunca se establece, por lo que las aplicaciones con logging básico **no registran el intento**.
3. **Resuelve DNS inverso** para el host objetivo.

5.2.1. Conteo de paquetes por puerto

- **Puerto abierto:** 3 paquetes — SYN (Nmap) + SYN/ACK (servicio) + RST (Nmap, aborta el handshake).
- **Puerto cerrado:** 2 paquetes — SYN (Nmap) + RST/ACK (kernel del host).
- **Puerto filtrado:** 1 paquete enviado + timeout sin respuesta.

5.3. Entorno de Pruebas

Para obtener evidencias reproducibles y éticamente correctas, todas las pruebas se han realizado sobre la interfaz de loopback (`lo`) de la propia máquina Kali Linux, sin generar tráfico hacia ningún sistema externo. Se levantaron dos servicios locales que actúan como objetivos legítimos:

- **Servidor HTTP en puerto 8080:** iniciado con `sudo python3 -m http.server 8080`. Este servicio crea un proceso en espacio de usuario que escucha activamente en el puerto 8080, respondiendo solicitudes HTTP. Desde el punto de vista del escáner, es indistinguible de un servidor web real.
- **Daemon SSH en puerto 22:** el servicio OpenSSH de Kali, activado con `sudo systemctl start ssh`. SSH es uno de los servicios más comunes en auditorías reales, por lo que su inclusión aporta representatividad a la evidencia.

La elección de `127.0.0.1` como objetivo garantiza que todo el tráfico permanece en la máquina local, cumpliendo con el aviso legal del enunciado que exige el uso de entornos simulados o virtualizados.

Antes de lanzar Nmap, se verificó que ambos servicios estaban escuchando correctamente mediante `ss -tlnp`, que confirmó el estado `LISTEN` en los puertos 22 y 8080. A continuación se arrancó Wireshark sobre la interfaz `lo` para capturar la totalidad del tráfico generado durante los tres escaneos.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -p 8080 127.0.0.1
sudo nmap -sS -p 22 127.0.0.1
sudo nmap -sS -p 9999 127.0.0.1
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 10:49 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00011s latency).

PORT     STATE SERVICE
8080/tcp open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 10:49 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000094s latency).

PORT   STATE SERVICE
22/tcp open  ssh

Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-25 10:49 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00019s latency).

PORT     STATE  SERVICE
9999/tcp closed abyss

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

Figura 3: Resumen de los tres escaneos ejecutados en secuencia desde la terminal. Nmap 7.98 identifica `8080/tcp open http-proxy`, `22/tcp open ssh` y `9999/tcp closed abyss`, demostrando los tres estados posibles de puerto en un único entorno de pruebas controlado.

5.4. Evidencias: Puerto Abierto 8080/tcp (HTTP)

Comando ejecutado: `sudo nmap -sS -p 8080 127.0.0.1`. Resultado: `8080/tcp open http-proxy`.

El servidor HTTP levantado con Python estaba escuchando activamente en el puerto 8080. En Wireshark (filtro `tcp.port == 8080`) se observan exactamente **3 paquetes**, que corresponden con el comportamiento característico del SYN Stealth Scan ante un puerto abierto:

1. Nmap envía un segmento `SYN` desde el puerto efímero 40599 hacia el puerto 8080, iniciando el proceso de handshake TCP.
2. El servidor HTTP responde con `SYN/ACK` desde el puerto 8080, señalizando que el puerto está abierto y hay un proceso dispuesto a aceptar la conexión.
3. Nmap, al recibir el SYN/ACK y confirmar el estado **open**, envía inmediatamente un `RST` para abortar el handshake sin completar la conexión. Esta es la característica definitoria del *half-open scan*: la conexión nunca llega a establecerse formalmente, por lo que el servidor no genera ningún registro de sesión.

La latencia entre el SYN inicial y el SYN/ACK de respuesta es de apenas 33 µs, lo que refleja la comunicación directa a través de la interfaz loopback sin salida a red física.

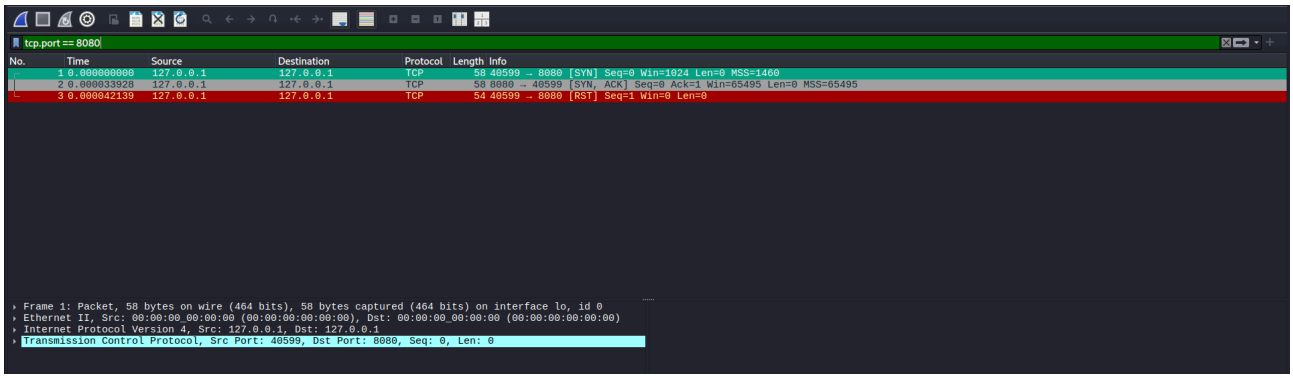


Figura 4: Evidencia Nmap — puerto **8080/tcp abierto**: secuencia `SYN → SYN/ACK → RST`. Terminal: `sudo nmap -sS -p 8080 127.0.0.1` ⟶ estado `open http-proxy`. Wireshark (filtro `tcp.port == 8080`): 3 paquetes. El SYN/ACK generado por el servidor HTTP Python en el paquete 2 es el indicador definitivo del estado **open**. Nmap cierra el intercambio con RST en el paquete 3, sin completar el handshake.

5.5. Evidencias: Puerto Abierto 22/tcp (SSH)

Comando ejecutado: `sudo nmap -sS -p 22 127.0.0.1`. Resultado: `22/tcp open ssh`.

El daemon OpenSSH estaba activo en el puerto 22. Wireshark (filtro `tcp.port == 22`) confirma exactamente el mismo patrón de 3 paquetes que en el caso anterior: `SYN → SYN/ACK → RST`. Este resultado demuestra que el comportamiento del SYN Stealth Scan es reproducible e independiente del servicio que escucha en el puerto: la secuencia de flags es idéntica para HTTP y para SSH, ya que el mecanismo opera a nivel de capa de transporte, antes de que el protocolo de aplicación entre en juego.

El daemon sshd responde con SYN/ACK en apenas 9 µs desde el puerto 22, tras lo cual Nmap emite el RST definitivo. La identificación del servicio como `ssh` la realiza Nmap consultando su base de datos `nmap-services`, sin necesidad de completar ninguna negociación de protocolo.

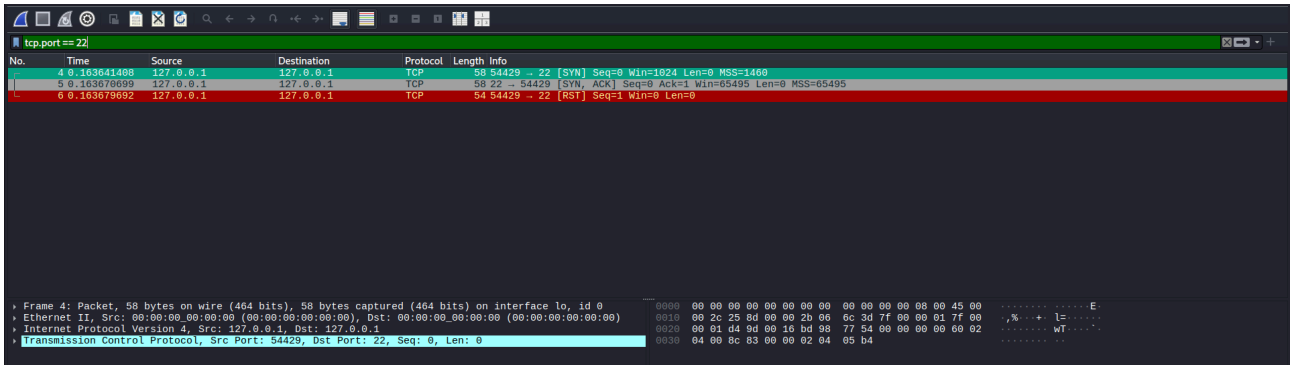


Figura 5: Evidencia Nmap — puerto **22/tcp abierto**: secuencia `SYN → SYN/ACK → RST`. Terminal: `sudo nmap -sS -p 22 127.0.0.1` ⟶ estado `open ssh`. Wireshark (filtro `tcp.port == 22`): 3 paquetes. El comportamiento es idéntico al del puerto 8080, evidenciando que el patrón SYN/ACK es universal para cualquier servicio activo, independientemente del protocolo de aplicación.

5.6. Evidencias: Puerto Cerrado 9999/tcp

Comando ejecutado: `sudo nmap -sS -p 9999 127.0.0.1`. Resultado: `9999/tcp closed abyss`.

El puerto 9999 no tiene ningún proceso asociado en el sistema. Wireshark (filtro `tcp.port == 9999`) muestra únicamente **2 paquetes**, una secuencia radicalmente diferente a la de los puertos abiertos:

1. Nmap envía el `SYN` desde el puerto efímero 35422 hacia el puerto 9999.

2. El kernel del sistema operativo, al no encontrar ningún socket escuchando en ese puerto, genera directamente un `RST/ACK` sin intervención de ninguna aplicación en espacio de usuario.

La ausencia del tercer paquete (el RST de Nmap) es el indicador clave: cuando el puerto está cerrado, es el propio kernel quien responde con RST/ACK y Nmap no necesita enviar nada más. La latencia de respuesta es de apenas 24 µs, ya que el kernel descarta la solicitud de inmediato.

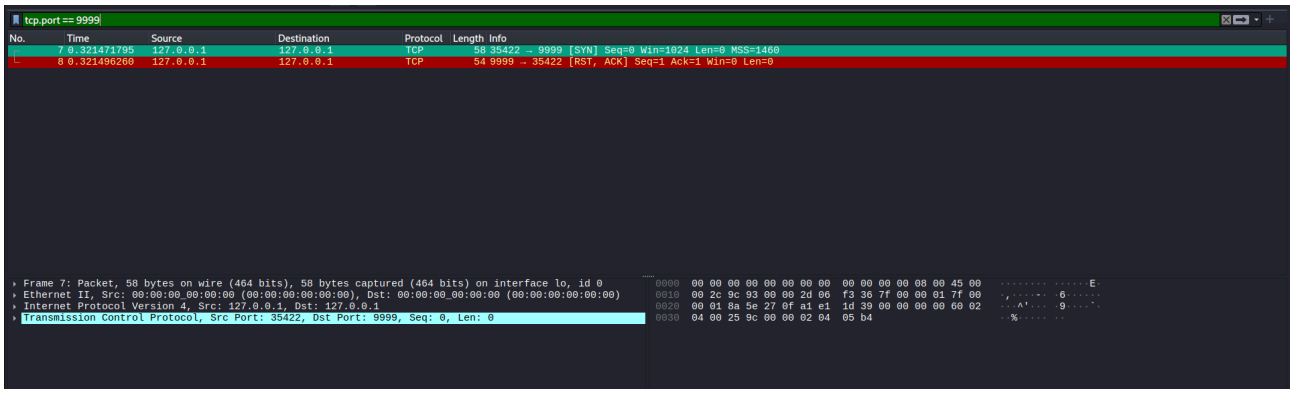


Figura 6: Evidencia Nmap — puerto **9999/tcp cerrado**: secuencia `SYN → RST/ACK`. Terminal: `sudo nmap -sS -p 9999 127.0.0.1` ⟶ estado `closed abyss`. Wireshark (filtro `tcp.port == 9999`): solo 2 paquetes. La ausencia de SYN/ACK es el indicador definitivo de que ningún proceso escucha en ese puerto. El RST/ACK es generado directamente por el kernel, no por ninguna aplicación.

5.7. Comparativa de los Tres Estados

La siguiente tabla sintetiza las diferencias observadas en las capturas entre los tres estados de puerto, consolidando los hallazgos de las evidencias anteriores:

Estado	Paquetes	Secuencia de flags	Quién responde
open (8080)	3	`SYN → SYN/ACK → RST`	Servicio en espacio de usuario
open (22)	3	`SYN → SYN/ACK → RST`	Daemon SSH (sshd)
closed (9999)	2	`SYN → RST/ACK`	Kernel directamente
filtered (12345)	1	`SYN → (timeout)`	Nadie — firewall descarta en silencio

La distinción operacional más relevante para un auditor es la siguiente: un puerto **open** siempre genera 3 paquetes y la respuesta proviene de un proceso en espacio de usuario; un puerto **closed** genera solo 2 paquetes y la respuesta la gestiona íntegramente el kernel; un puerto **filtered** genera únicamente 1 paquete enviado y ninguna respuesta, siendo el estado más lento de confirmar ya que Nmap debe esperar a que expire el timeout (aproximadamente 1 segundo por defecto).

5.8. Evidencias: Puerto Cerrado 80/tcp

Comando ejecutado: `sudo nmap -sS -p 80 127.0.0.1`. Resultado: `80/tcp closed http`.

En Wireshark (filtro `tcp.port ==80`) se observan exactamente **2 paquetes**:

1. Nmap envía `SYN` desde el puerto efímero 62416 hacia el puerto 80.
2. El kernel responde con `RST, ACK` desde 80, confirmando el estado **closed**.

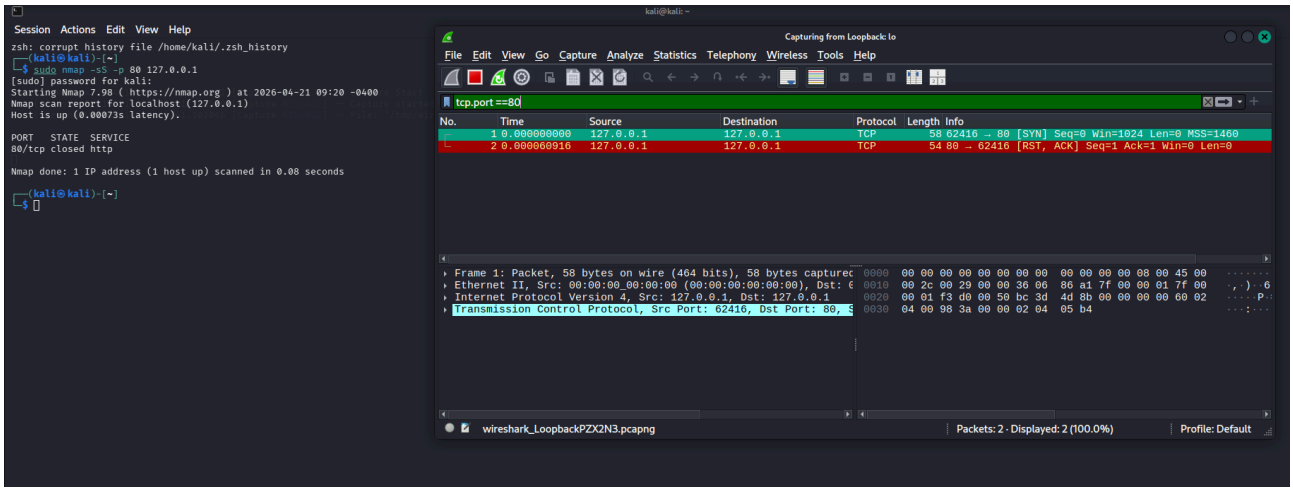


Figura 7: Evidencia Nmap — puerto **80/tcp cerrado**: secuencia `SYN → RST/ACK`. Terminal: `sudo nmap -sS -p 80 127.0.0.1` ⟶ estado `closed`. Wireshark: 2 paquetes. El RST/ACK es generado directamente por el kernel al no existir servicio en el puerto 80, con latencia de apenas 60 µs.

5.9. Evidencias: Puerto Cerrado 1234/tcp

Comando ejecutado: `sudo nmap -sS -p 1234 127.0.0.1`. Resultado: `1234/tcp closed hotline`.

Wireshark (filtro `tcp.port ==1234`) confirma el mismo patrón: **SYN ⟶ RST/ACK**, demostrando la reproducibilidad del comportamiento con cualquier puerto sin servicio activo.

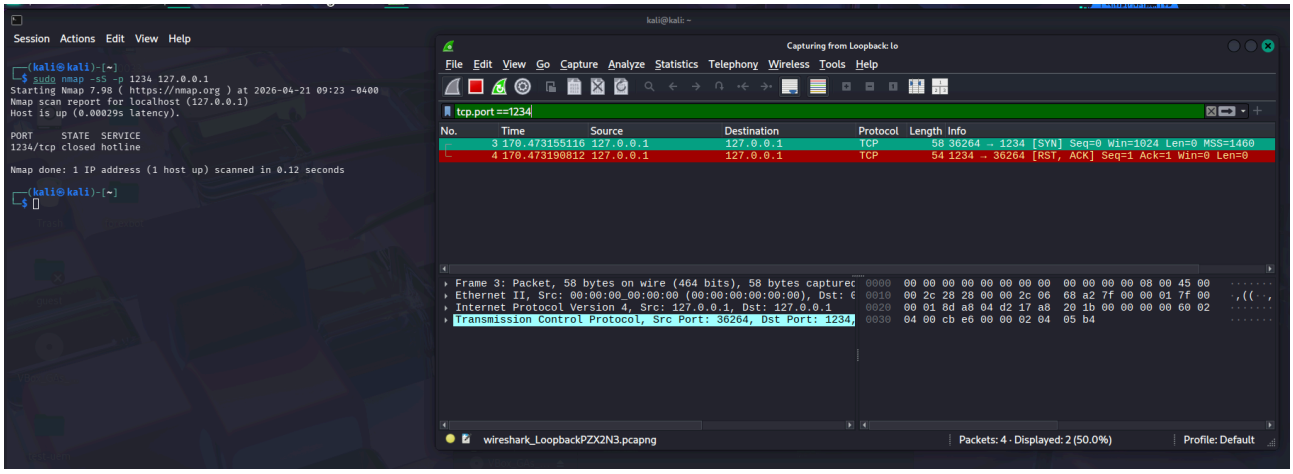


Figura 8: Evidencia Nmap — puerto **1234/tcp cerrado**: secuencia `SYN → RST/ACK`. Puerto no estándar que confirma la generalidad del comportamiento. Nmap etiqueta el servicio como `hotline` según su base de datos `nmap-services`. La latencia de respuesta RST es igualmente sub-milisegundo, gestionada íntegramente por el kernel sin intervención de ninguna aplicación en espacio de usuario.

La distinción entre ambos estados es operacionalmente relevante: un puerto **closed** implica que el host responde activamente con RST/ACK con latencia inferior a 1 ms, lo que aporta certeza total sobre la existencia del host y la ausencia de servicio. Un puerto **filtered**, en cambio, no genera respuesta hasta que expira el timeout (aproximadamente 1 segundo por defecto), resultando ambiguo: puede deberse tanto a la inexistencia del host como a un firewall que descarta el tráfico silenciosamente. Este comportamiento hace que los puertos filtrados sean significativamente más lentos de confirmar durante una auditoría.

5.10. Evidencias: Puerto Filtrado 12345/tcp

Comando ejecutado: `sudo nmap -sS -p 12345 --max-retries 0 127.0.0.1`. Resultado: `12345/tcp filtered netbus`.

Para simular el estado **filtered** de forma reproducible y ética sobre la propia máquina, se insertó una regla de firewall mediante `iptables` que descarta silenciosamente todo tráfico TCP entrante en el puerto 12345:

```
sudo iptables -A INPUT -p tcp --dport 12345 -j DROP
```

Esta regla hace que el kernel descarte el paquete SYN antes de que ninguna aplicación lo procese, sin enviar ninguna respuesta al emisor. Wireshark (filtro `tcp.port == 12345`) muestra únicamente **1 paquete**:

1. Nmap envía el `SYN` desde el puerto efímero 44712 hacia el puerto 12345.
2. No se recibe ninguna respuesta. Nmap espera hasta que expira el timeout y clasifica el puerto como **filtered**.

El parámetro `--max-retries 0` se utilizó para forzar un único intento de SYN, eliminando los reintentos automáticos de Nmap y obteniendo así la captura más limpia posible. El aviso explícito de Nmap en la terminal — *«giving up on port because retransmission cap hit (0)»* — confirma que la ausencia de respuesta es total y deliberada.

La duración del escaneo fue de **1.06 segundos**, contrastando radicalmente con los 0.08–0.11 segundos de los puertos cerrados. Esta diferencia de más de un orden de magnitud es la consecuencia directa del timeout: el escáner no puede distinguir entre un host inexistente y un firewall que filtra silenciosamente, por lo que debe esperar hasta agotar el tiempo de espera antes de emitir su veredicto.

Una vez obtenidas las evidencias, la regla de firewall fue eliminada para restaurar el estado original del sistema:

```
sudo iptables -D INPUT -p tcp --dport 12345 -j DROP
```

```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ sudo iptables -A INPUT -p tcp --dport 12345 -j DROP
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -p 12345 --max-retries 0 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-26 12:47 -0400
Warning: 127.0.0.1 giving up on port because retransmission cap hit (0).
Nmap scan report for localhost (127.0.0.1)
Host is up.

PORT      STATE    SERVICE
12345/tcp filtered netbus

Nmap done: 1 IP address (1 host up) scanned in 1.06 seconds

┌──(kali㉿kali)-[~]
└─$
```

Figura 9: Evidencia Nmap — puerto **12345/tcp filtrado**: secuencia `SYN → (timeout)`. Terminal: `sudo nmap -sS -p 12345 --max-retries 0 127.0.0.1` ⟶ estado `filtered netbus`. La duración de 1.06 s frente a los 0.08 s de un puerto cerrado evidencia el coste temporal del timeout. El aviso *«giving up on port because retransmission cap hit (0)»* confirma la ausencia total de respuesta.

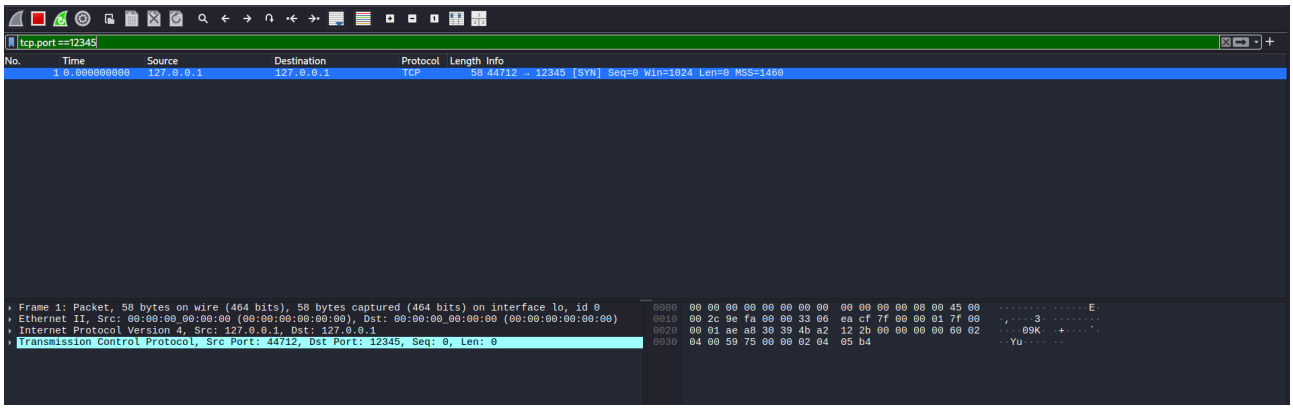


Figura 10: Wireshark — puerto **12345/tcp filtrado**: filtro `tcp.port == 12345`. Un único paquete SYN visible (paquete 1, sin respuesta), frente a los 3 paquetes del estado **open** y los 2 del estado **closed**. La ausencia de cualquier respuesta es la firma inequívoca del filtrado por firewall.

5.11. Resumen del Comportamiento por Defecto

Parámetro	Valor por defecto	Flag para cambiar
Técnica de escaneo	SYN Stealth (`-sS`)	`-sT`, `-sU`, `-sA`...
Puertos escaneados	Top 1000 de `nmap-services`	`-p-`, `-p 1-65535`
Paquetes por puerto (open)	3 (SYN + SYN/ACK + RST)	—
Paquetes por puerto (closed)	2 (SYN + RST/ACK)	—
Paquetes por puerto (filtered)	1 (SYN + timeout)	—

Resolución DNS	Activada	`-n` para desactivar
Timing	Normal (`-T3`)	`-T0` (lento) a `-T5` (agresivo)

6. Conclusiones

Esta práctica ha permitido consolidar los conocimientos teóricos sobre TCP/IP mediante su aplicación práctica directa, tanto en la construcción manual de paquetes con Scapy como en el análisis del comportamiento de Nmap a través de capturas de Wireshark.

1. **Scapy como herramienta de bajo nivel:** La construcción manual de paquetes proporciona una comprensión profunda de las cabeceras de protocolo. La función `craft_discovery_pkts` demuestra que con pocas líneas de Python es posible replicar comportamientos similares a los de escáneres industriales.
2. **Complementariedad de los tres protocolos:** TCP ACK, UDP e ICMP Timestamp cubren vectores de detección distintos y complementarios. La combinación reduce los falsos negativos frente a políticas de filtrado heterogéneas.
3. **SYN Stealth Scan de Nmap:** El análisis de las capturas confirma que Nmap implementa un half-open scan preciso y reproducible. La secuencia `SYN → SYN/ACK → RST` para puertos abiertos y `SYN → RST/ACK` para puertos cerrados ha sido verificada empíricamente sobre servicios reales (HTTP y SSH), demostrando que el patrón es universal e independiente del protocolo de aplicación.
4. **Importancia de los servicios activos como evidencia:** Las pruebas sobre puertos abiertos (8080 y 22) frente a puertos cerrados (9999, 80, 1234) permiten observar directamente la diferencia entre una respuesta SYN/ACK generada por un proceso en espacio de usuario y un RST/ACK generado directamente por el kernel, lo que ilustra de forma práctica la arquitectura del stack TCP/IP de Linux.
5. **Estado filtrado y coste del timeout:** La simulación del estado **filtered** mediante `iptables` evidencia que la ausencia de respuesta es la firma del filtrado silencioso. El tiempo de escaneo de 1.06 s frente a los 0.08 s de un puerto cerrado ilustra por qué los puertos filtrados penalizan significativamente la velocidad de una auditoría real, y por qué herramientas como Nmap implementan mecanismos de paralelización agresiva para mitigar este efecto.
6. **Perspectiva dual:** Conocer las técnicas de reconocimiento activo es inseparable de entender cómo detectarlas. Los patrones de tráfico observados son exactamente los que un IDS como Snort o Suricata utilizaría para identificar un escaneo en un entorno de producción.

7. Referencias Bibliográficas

- Lyon, G. (2009). *Nmap Network Scanning*. Insecure.Com LLC. https://nmap.org/book/
- Postel, J. (1981). *RFC 792: Internet Control Message Protocol*. IETF.
- Postel, J. (1981). *RFC 793: Transmission Control Protocol*. IETF.
- Postel, J. (1980). *RFC 768: User Datagram Protocol*. IETF.
- Scapy Project. (2024). *Scapy Documentation*. https://scapy.readthedocs.io
- Wireshark Foundation. (2024). *Wireshark User's Guide*. https://www.wireshark.org/docs/